Isolating Compiler Faults Through Differentiated Compilation Configurations

Yibiao Yang, Qingyang Li, Maolin Sun, Jing Yang, Jiangchang Wu, and Yuming Zhou

***Abstract*—Compilation optimization bugs are prevalent and can significantly affect the correctness of software products, posing serious challenges to software development. Identifying and localizing these bugs are critical tasks for compiler developers. However, the intricate nature and extensive scale of modern compilers make it difficult to pinpointing the root causes of such bugs. Previous research has introduced innovative techniques that generate *witness test programs*–tests that pass–by mutating bug-triggering test cases, highlighting the importance of this problem and demonstrating the effectiveness of such approaches. Nevertheless, existing techniques based on witness test programs generation suffer from inherent limitations. Specifically, they do not guarantee the successful creation of witness test programs via mutation and are often time-consuming, typically requiring extensive iterations to produce a valid witness test program. In this study, we present ODFL, a simple yet effective approach for automatically isolating compiler optimization faults by introducing the concept of *differentiated compilation configurations*. The core insight behind ODFL is that modifying compilation settings such as disabling fine-grained compilation flags in GCC or reducing the number of fine-grained compilation passes in LLVM, can suppress the manifestation of compiler bugs triggered by the same test program. Through adjusting these settings, ODFL creates differentiated compilation configuration that produce multiple compiler executions with distinct pass/fail outcomes. We utilize these differentiated configurations to collect both passing and failing compiler coverage, and then apply *Spectrum-Based Fault Localization (SBFL)* techniques to rank compiler source files based on their suspiciousness. Our evaluation of 60 GCC and 50 LLVM compiler bugs demonstrates that ODFL substantially outperforms state-of-the-art compiler fault localization techniques in terms of both effectiveness and efficiency. Notably, ODFL achieves over 90% improvement in accurately ranking the top-1 faulty source files compared to three existing techniques–DiWi, RecBi, and LLM4CBI–and reduces fault localization time by more than 99% on average.**

***Index Terms*—Compilers, fault localization, compilation configuration, fine-grained options.**

Received 3 September 2024; revised 26 April 2025; accepted 4 May 2025. Date of publication 13 May 2025; date of current version 16 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 624B2067, Grant 62072194, and Grant 62172205; in part by Jiangsu Natural Science Foundation under Grant BK20231402; in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization; and in part by the Fundamental Research Funds for the Central Universities under Grant 14380121. Recommended for acceptance by T. Hall. *(Corresponding authors: Qingyang Li; Maolin Sun; Yuming Zhou.)*

The authors are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: yangyibiao@nju.edu.cn; liqingyang@smail.nju.edu.cn; merlin@smail.nju.edu.cn; njuyj@smail.nju.edu.cn; jiangchangwu@smail.nju.edu.cn; zhouyuming@nju.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3569530

I. INTRODUCTION

COMPILERS are a cornerstone of modern software development, responsible for translating source code written in high-level programming languages into machine code executable by computer systems. Despite their essential role in the software development lifecycle, compilers, being complex software systems, are prone to errors, with numerous bugs reported daily. Complicating the issue further, modern compilers incorporate a wide range of advanced code optimizations aimed at improving the performance and reducing the size of the generated binaries. Previous research has shown that a considerable proportion of compiler bugs are associated with these optimizations [1]. For instance, our analysis of the defects reported in the GCC Bugzilla database reveals that, to date, over 7,000 resolved defects are related to the tree optimization component. This underscores the importance of effectively isolating faults introduced by compiler optimizations. Such bugs are particularly critical, as they can compromise the correctness of virtually all compiled software systems compiled using the affected compiler. However, the inherent complexity and scale of compilers present significant challenges in pinpointing the exact code elements responsible for these defects.

This important question was initially raised by Chen et al. [2], [3], who subsequently introduced pioneering techniques for isolating compiler faults by generating witness test programs. More recently, Tu et al. [4] proposed leveraging the capabilities of Large Language Models (LLMs) to generate such witness programs. Both approaches involved generating the witness test programs through random mutations of bug-triggering test programs. A bug-triggering program is specifically designed to expose faults in the compiler, while a witness program is intentionally generated to avoid triggering the same fault under identical compilation configurations. By combining the bug-triggering and corresponding witness test programs, Spectrum-Based Fault Localization (SBFL) techniques can be employed to calculate the suspiciousness of individual compiler files for fault localization [5], [6]. SBFL techniques utilize the coverage information of both passing and failing execution traces to assess the suspiciousness of faulty code. The underlying concept is to compare the execution traces of failing test cases, which expose the fault, with those of passing test cases, which do not, thereby identifying the program

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

entities most likely responsible for the fault [5], [7]. In the context of compiler fault localization, the coverage of passing traces refers to the compiler's coverage while compiling a witness test program, whereas the coverage of failing traces pertains to the compiler's coverage when compiling a bug-triggering test program. While existing witness-based approaches can effectively locate compiler bugs, they may have inherent limitations. Firstly, there is no guarantee that witness test programs can always be generated through random mutations of bug-triggering test programs. Secondly, generating witness programs can be a time-consuming process that necessitates numerous mutation attempts. Lastly, the localization results derived from witness test programs may be unreliable, particularly due to the random mutation strategies employed.

Approach. In this study, we present a novel approach, ODFL (**O**ption **D**ifferentiating for **F**ault **L**ocalization), designed to address the challenges of effectively and efficiently isolating compiler faults, especially optimization-related bugs. ODFL involves exploring differentiated compilation configurations, which consist of both failing and passing configurations that differ minimally in fine-grained compilation options. In these differentiated compilation configurations, a compiler bug manifests when the compiler compiles the bug-triggering test program using the failing configuration, while the bug is concealed when the same test program is compiled using the passing configuration. Our goal is to systematically identify these differentiated compilation configurations. The key observation underpinning the development of ODFL is that even minor modifications to the compilation configuration—such as disabling specific fine-grained compilation options in the GNU Compiler Collection (GCC) or reducing the number of compilation passes in LLVM—can effectively conceal the compiler fault initially exposed by the bug-triggering test program. These modifications lead to the creation of what we term "differentiated compilation configurations." Such configurations allow us to obtain both failing and passing coverage of code in compilers. Based on the coverage information, suspicious scores for the code elements can be calculated using SBFL formulas to identify potentially faulty files in compilers. Our comprehensive evaluation of ODFL, utilizing a sample size of 60 real GCC bugs and 50 real LLVM bugs, demonstrates its effectiveness in accurately localizing compiler bugs. The results indicate that ODFL outperforms existing compiler fault localization approaches by successfully isolating 31, 56, and 73 compiler bugs within the Top-1, Top-5, and Top-10 files, respectively. Notably, ODFL significantly outperforms all existing witness-based compiler fault localization techniques—DiWi [2], RecBi [3], and LLM4CBI [4]—by isolating about 100% more bugs within the Top-1 files.

Contributions. We make the following major contributions:

- **Concept:** We introduce the novel concept of differentiated compilation configurations, consisting of failing and passing configurations. Failing configurations adhere to the reduction principle, enabling a minimal set of fine-grained compilation options, while passing configurations adhere to the alignment principle, closely resembling the failing configurations. Thus, these differentiated configurations exhibit minimal disparities in fine-grained options.
- **Novelty:** We propose ODFL, a spectrum-based compiler fault isolation approach leveraging differentiated compilation configurations. ODFL serves as an effective tool for isolating compiler faults, significantly enhancing the effectiveness and efficiency of existing techniques by incorporating these configurations.
- **Implementation:** We have meticulously implemented ODFL as a practical fault localization tool, ensuring its ease of extension and usability in real-world scenarios. This practical implementation is of paramount importance, as it enhances the tool's usability and applicability for developers and researchers in various application scenarios.
- **Evaluation:** To evaluate the effectiveness and efficiency of our proposed approach, we conducted extensive experiments on comprehensive sets of 60 GCC bugs and 50 LLVM bugs, demonstrating the wide applicability of our approach. The experiments demonstrate ODFL's effectiveness and efficiency, providing concrete evidence of its capabilities in practical compiler fault isolation.

This journal version significantly extends our earlier work published in the proceedings of SANER 2022 [8], incorporating three major enhancements:

- ⋆ **Systematic Construction of Differentiated Compilation Configurations.** We propose a more structured and efficient methodology for deriving differentiated compiler configurations. Starting from a coarse-grained failing configuration, our journal extension introduces a progressive deactivation strategy for fine-grained optimizations. This approach iteratively disables fine-grained optimization flags or optimization passes to generate configurations with the minimum number of fine-grained optimizations enabled. From this minimized failing configuration, we further deactivate fine-grained optimizations to derive corresponding passing configurations. Compared to the conference version, this strategy significantly reduces the time required to generate differentiated compilation configurations over the conference version, making ODFL more practical for integration into real-world development pipelines.
- ⋆ **Support for Multiple Compiler Architectures.** The conference version of ODFL was limited to GCC and could not be directly applied to LLVM due to the lack of official documentation mapping optimization passes to levels (e.g., O0, O1, ..., O3) and the absence of mechanisms for selectively disabling individual passes. The journal version extends ODFL to support LLVM by integrating LLVM's built-in `opt-bisect-limit` feature with a binary search over the passes. This integration enables the identification of the threshold at which pass begin to trigger failures. Consequently, ODFL can now generate differentiated configurations tailored to the LLVM architecture, facilitating automated spectrum-based fault localization for LLVM.
- ⋆ **Comprehensive Empirical Evaluation.** We substantially expanded our evaluation in two key dimensions. First, to assess robustness, we conducted experiments that progressively disabling optimization options in GCC with

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

randomized orders. ODFL consistently maintained high localization accuracy across these runs, demonstrating strong resilience to order variations and confirming its stability across different settings. Second, we broadened our dataset to include 110 real-world compiler bugs, 60 from GCC and 50 from LLVM. Our analysis includes detailed performance breakdowns and statistical comparisons. Results show that ODFL substantially outperforms prior techniques (DiWi, RecBi, and LLM4CBI), achieving over a 99% reduction in localization time.

Organization. The rest of this paper is organized as follows. Section II introduces the background of compiler compilation options and the Spectrum-based Fault Localization technique. Section III presents the motivation behind our approach. Section IV elaborates on our approach in detail. Then, our experimental setup and results are presented in Section V. After that, we present detailed discussions regarding our study in Section VI. Finally, the related works are surveyed in Section VII and the conclusion is in Section VIII.

II. BACKGROUND

In this section, we first briefly describe the Spectrum-Based Fault Localization (SBFL) technique utilized in our study. We then introduce the background of compiler compilation options and compiler faults. Finally, we summarize existing compiler fault isolation techniques and discuss their limitations.

A. Spectrum-Based Fault Localization

Spectrum-Based Fault Localization (SBFL) is a prominent technique in software testing and debugging, designed to identify the root cause of software faults or failures [5], [7]. The fundamental principle of SBFL is to strategic utilization of coverage information derived from the executed code to assess the suspiciousness of individual code statements or blocks, thereby ranking them based on their likelihood of being faulty [5]. This approach is grounded in the assumption that code statements or blocks executed more frequently by failing test cases are more likely to be associated with the underlying fault. By employing SBFL, developers can enhance the efficiency of their debugging efforts by concentrating on the most probable fault locations. The effectiveness of SBFL is significantly influenced by the disparities between failing and passing coverage, as it relies on the comparative analysis of these program spectra [7], [9], [10]. Consequently, minimizing the differences between failing and passing coverage can substantially improve the efficacy of SBFL techniques. This underscores the importance of effective passing and failing execution traces within the SBFL framework.

B. Compiler Options

Compilation process is typically characterized by various compilation levels, which, in the context of compilers such as GCC and LLVM, are often coarse-grained, labeled as "-O0", "-O1", "-O2", and "-O3". These levels come bundled with predefined fine-grained compilation options, carefully chosen to balance code execution speed and compilation time, thus achieving optimized output. In addition to these coarse-grained levels, compilers provide a range of fine-grained compilation options, granting developers enhanced control to tailor their compilation strategies to specific needs. These fine-grained options are diverse, including but not limited to, loop unrolling, function inlining, common subexpression elimination, dead code elimination, and register allocation. For instance, in GCC, function inlining can be enabled or disabled using the flags "-finline-functions" and "-fno-inline-functions", respectively. This option is enabled by default at GCC's "-O2", "-O3", and "-Os" levels[1]. Similarly, in LLVM, developers can utilize the "-opt-bisect-limit" option to specify the number of compilation passes during compilation[2]. This capability allows for precise tuning of compilation processes, offering developers the flexibility to align their code with performance goals using the most effective compilation strategies.

C. Compiler Faults

In the landscape of software development, the concept of compiler faults holds significant importance due to their impact on program performance and correctness [11]. Compilers are designed to enhance the efficiency and performance of code, potentially increasing speed, reducing memory usage, and improving overall execution time [12], [13]. However, it is essential to recognize that the compilation phase or the compilation passes can introduce bugs into the compiled code [11]. These faults can lead to unexpected or erroneous behavior in the compiled output, stemming from various causes, including inherent issues within compilation algorithms or mishandling of specific code patterns. The consequences of such faults can pose challenges to software reliability and security, manifesting as incorrect program outputs, system crashes, or the creation of security vulnerabilities. They may also arise from interactions between different compilation techniques employed by the compiler. The complexity of optimizing code, coupled with the diverse nature of programming languages and compiler architectures, increases the likelihood of such faults occurring. An empirical study analyzing compiler bugs within a bug tracking system reveals that compiler faults account for a significant proportion of reported compiler issues [14]. This finding underscores the critical nature of understanding and mitigating these faults to ensure software reliability, security, and overall quality.

D. Compiler Fault Localization

The field of compiler fault localization presents significant challenges, primarily due to the intricate nature of compilers and the vast array of compilation options available to developers. Compilers are responsible for numerous options that significantly enhance program efficiency. However, this complexity complicates the task of understanding the interplay of various compilation options and pinpointing the root cause of bugs. Moreover, localizing and diagnosing bugs can be particularly time-consuming, given the multitude of compilation options

[1]https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html
[2]https://llvm.org/docs/OptBisect.html

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

that developers must consider. Chen et al. propose a promising solution to these challenges through a witness test program-based approach [2], [3]. Unlike bug-triggering test programs, which are designed to manifest compiler bugs, witness test programs-obtained through program mutation-do not trigger corresponding compiler bugs. Recently, Tu et al. [4] proposed harnessing the power of Large Language Models (LLMs) to generate witness test programs. These innovative approaches provide a more nuanced method for locating compiler bugs, as witness test programs offer additional insights that traditional methods may overlook. This, in turn, aids in accurately localizing and diagnosing compiler faults. The witness test program-based approach holds significant promise in addressing the existing challenges in compiler fault localization. By generating these witness test programs, both passing and failing test coverage can be obtained. This information, when integrated into Spectrum-Based Fault Localization (SBFL) techniques, facilitates accurate compiler bug localization and diagnosis, offering a comprehensive solution to the complexities associated with compiler fault localization.

E. Discussion on Challenges in Existing Techniques

The pioneering work of Chen et al. [2], [3] introduced witness test program-based techniques as an innovative approach to the fundamental challenges of compiler fault localization. These methods represent a significant advancement in the field, emphasizing the necessity of addressing this critical problem directly. The effectiveness of existing witness test program-based techniques in accurately identifying compiler bugs has been demonstrated through extensive evaluation. However, it is important to acknowledge certain inherent limitations of these techniques. One notable limitation is the time-consuming nature of generating witness test programs. This process often requires numerous attempts to obtain a mutated program that successfully passes the test, which can be cumbersome and inefficient, particularly for large and complex programs. Additionally, potential reliability concerns arise, as localization results-critical for accurate bug identification-may be compromised due to the inherent randomness of program generation process. This randomness can lead to inconsistent and unreliable outcomes, thereby casting doubt on the accuracy of the localization results. Consequently, the results from witness test program-based approaches may not consistently provide accurate outcomes, limiting their practical applicability in the compiler fault localization.

III. Motivation

In this section, we first use two concrete examples to illustrate the motivation behind our approach. Then, we analyze the impact of the number of fine-grained compilation options involved in compilation on the code coverage of compilers.

A. Motivating Example

When reporting a compiler fault, it is essential to provide both the bug-triggering test program and the corresponding

```c
#include <stdio.h>
int a, b, c;
short f(int x, int y) {
  return x + y;
}
int main() {
  a = 0;
  for(; a<30; a=f(a, 4)) {
    c = b; b = 6;
  }
  printf("%d\n", c);
  return 0;
}
```

(a) Bug-triggering program g.c

```
$ gcc -O2 g.c && ./a.out
6
$ gcc -O3 g.c && ./a.out
0
$ gcc -O3 -fno-tree-loop-vectorize g.c & ./a.out
6
```

(b) Compilation configuration

Fig. 1. GCC bug #71439. When the test program is compiled at the "-O2", it correctly outputs "6". However, at the "-O3", it incorrectly outputs "0". When the fine-grained option "-ftree-loop-vectorize" is disabled under "-O3", the program correctly outputs "6" once again. This indicates that the compilation configuration has shifted from failing to passing.

compilation configuration. This comprehensive information enables accurate bug reproduction, facilitating effective bug localization and resolution. The manifestation of bugs depends on the characteristics of the test program and the employed compilation configuration, both of which serve as critical inputs to the compiler. Existing compiler bug localization techniques primarily focus on mutating the original bug-triggering test program to obtain witness test programs that obscure the bug. However, we explore the aspect of compilation configuration, aiming to isolate compiler bugs using differentiated compilation settings. Encouragingly, we have observed that minor modifications to the compilation configuration, such as disabling specific fine-grained compilation options in GCC or reducing compilation passes in LLVM, can effectively conceal compiler bugs initially exposed by the bug-triggering test program. To illustrate this, we present two concrete examples of compiler bugs within GCC and LLVM compilers:

- **GCC Bug:** Fig. 1 depicts the real GCC bug #71439. Fig. 1(a) represents the reported bug-triggering test program, while Fig. 1(b) displays the outputs of the program compiled under different compilation configurations. Notably, the program produces the expected output of "6" when compiled with the "-O2" level but yields an incorrect output of "0" with "-O3", indicating a bug in GCC. Interestingly, by disabling loop vectorization on trees using the fine-grained option "-fno-tree-loop-vectorize", the program produces the expected output of "6", highlighting the effectiveness of minor modifications to the compilation configuration in concealing the bug initially exposed by the test program.
- **LLVM Bug:** Fig. 2 illustrates the real LLVM bug #61312. As demonstrated in Fig. 2(b), the test program behaves as

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

```c
int main() {
  int a;
  for (a = 2; a >= 0; a--)
    ;
  unsigned b = -1 % a;
  a = b;
  if (a != 0)
    __builtin_abort ();
  return 0;
}
```

(a) Bug-triggering program v.c

```
$ clang -O0 v.c && ./a.out
$ clang -O1 v.c && ./a.out
Aborted
$ clang -O1 -mllvm -opt-bisect-limit=6 v.c && ./a.out
$ clang -O1 -mllvm -opt-bisect-limit=7 v.c && ./a.out
Aborted
```

(b) Compilation configuration

Fig. 2. LLVM bug #61312. When the test program is compiled at the "-O0", it executes correctly and does not abort. However, when compiled at "-O1", the program's behavior becomes incorrect and it aborts. Setting the value of "-opt-bisect-limit" to 6 under "-O1"-which disables all compilation passes above the specified limit-allows the program to run without aborting. Conversely, increasing the value of "-opt-bisect-limit" from 6 to 7 causes the program to abort again. This indicates a transition from a passing to a failing state.

expected when compiled with the "-O0" level but terminates unexpectedly with "-O1", signifying a bug within LLVM. Notably, setting the value of "-opt-bisect-limit" to 6 restores expected behavior. However, setting it to 7 results in abnormal termination. These observations further underscore the significance of fine-tuning the compilation configuration to effectively conceal bugs exposed by the same test program.

These two examples clearly illustrate that minor modifications to the compilation configuration can effectively conceal the same bug originally exposed by an identical bug-triggering test program.

B. Differentiated Compilation Configurations

Based on the observations in Fig. 1 and Fig. 2, we recognize that even slight differences in compilation configurations can lead to significant variations in the execution results or behaviors of the same test program. This finding suggests that the outcomes of the test program can oscillate between failing and passing, depending on the compilation configurations employed. To facilitate further discussion, we introduce the following definitions:

- **Failing Compilation Configuration:** A failing compilation configuration refers to a compiler compilation setting that triggers a compiler bug when applied to a given test program. In other words, when this configuration is used to compile the bug-triggering test program, the test program exhibits unexpected behavior.
- **Passing Compilation Configuration:** A passing compilation configuration denotes a compilation setting that does not trigger the compiler bug when applied to the same test program. Specifically, when this compilation configuration is used to compile the bug-triggering test program, it behaves as expected.

The failing and passing compilation configurations collectively form a set of differentiated compilation configurations. For instance, in Fig. 1(b), the configurations "gcc -O2 g.c" and "gcc -O3 -fno-tree-loop-vectorize g.c" are considered passing compilation configurations, while "gcc -O3 g.c" is regarded as a failing configuration. Similarly, in Fig. 2(b), "clang -O0 v.c" and "clang -O1 -mllvm -opt-bisect-limit=6 v.c" are classified as passing compilation configurations, whereas "clang -O1 v.c" and "clang -O1 -mllvm -opt-bisect-limit=7 v.c" are classified as failing configurations.

C. Impact of Fine-Grained Compilation Options on Coverage

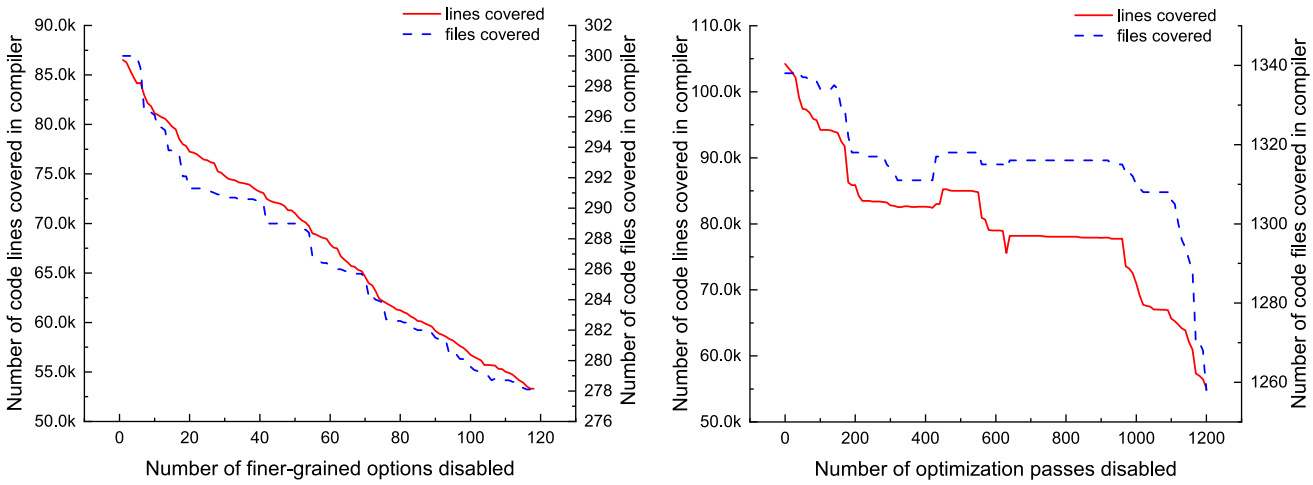
As previously mentioned, each coarse-grained compilation, such as the "-O3" level, comprises a collection of fine-grained compilation options or passes. In the GCC compiler, individual fine-grained compilation options can be disabled using the "-fno-opt" flag, where "opt" represents a specific fine-grained compilation option. Similarly, in LLVM, fine-grained compilation passes can be disabled using the "-opt-bisect-limit" flag. By specifying "-opt-bisect-limit=n", all compilation passes above a specific limit "n" are disabled during compilation in LLVM. In our proposed approach to locating compiler faults based on differentiated compilation configurations, it is crucial to understand the relationship between the number of disabled fine-grained compilation options and the resulting code coverage.

To evaluate the potential effectiveness of our approach, we examined the relationship between the number of disabled fine-grained compilation options or passes and the code coverage in two compiler revisions: GCC revision 199531 and LLVM revision of commit ab8d7ea.[3] Fig. 3(a) depicts the variation in the number of covered lines and files as the number of disabled fine-grained compilation options increases in GCC. To obtain the data, we conducted 10 independent runs, each time disabling the fine-grained compilation options in a random order and averaging the results. Similarly, Fig. 3(b) illustrates the change in code coverage as the number of disabled fine-grained compilation passes increases in LLVM. The X-axes in both figures represent the number of disabled fine-grained compilation options or passes, while the left Y-axis indicates the number of covered lines and the right Y-axis indicates the number of covered files in the compilers. Note that for improved visualization, the starting point of the y-axis in both Fig. 3(a) and Fig. 3(b) has been adjusted away from zero.

Fig. 3(a) reveals that in GCC, the number of covered lines decreases from approximately 86,000 to about 53,000, and the number of covered files reduces from around 300 to about 280 files when the number of disabled fine-grained compilation options increases from 1 to 120. Similarly, Fig. 3(b) illustrates that in LLVM, the number of covered lines decreases from

[3]https://github.com/llvm/llvm-project/commit/ab8d7ea4862805077e1606f587c82182b6060068

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.



(a) GCC (b) LLVM

Fig. 3. Impact of fine-grained compilation options on code coverage of compilers. The y-axis representing the quantity of line coverage begins at 50K, while the y-axis representing the quantity of file coverage starts at 276 and 1260, respectively.

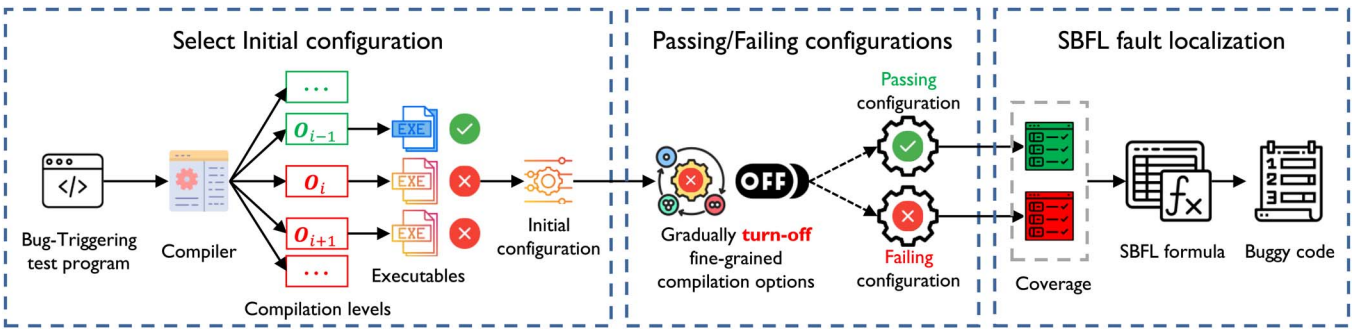


Fig. 4. An overview of ODFL.

approximately 105,000 to about 55,000, and the number of covered files reduces from about 1,340 to around 1,120 files as the number of disabled fine-grained compilation passes increases from 1 to 1,200.

The findings from Fig. 3(a) and 3(b) indicate that disabling more fine-grained compilation options or passes results in fewer code elements involved in compilation for both GCC and LLVM. This suggests that, for failing configurations, disabling more fine-grained compilation options or reducing the number of compilation passes is advantageous. This strategy can eliminate more code elements that are not responsible for triggering the compiler bug, thereby facilitate subsequent compiler fault localization.

IV. APPROACH

In this section, we provide an overview of ODFL, elaborate on our methodology, and present the Spectrum-Based Fault Localization (SBFL) formula employed within ODFL. Fig. 4 illustrates our framework.

A. Overview

ODFL encompasses three key steps. The first step involves selecting the lowest level that reproduces the bug as the initial compilation configuration. The second step entails searching for differentiated compilation configurations based on the initial configuration. The third step calculates suspicious scores for code entities using SBFL formulas, leveraging the code coverage spectra obtained from the differentiated compilation configurations.

As discussed in Section III, disabling more fine-grained compilation options reduces the compiler code involved in the compilation process, thereby eliminating code elements that are not responsible for triggering the compiler bug. This reduction facilitates subsequent compiler bug localization. Consequently, one design principle of our proposed approach, ODFL, is to disable as many fine-grained compilation options as possible in the failing compilation configuration, resulting in fewest fine-grained compilation options enabled. Additionally, another important design aspect of ODFL is to generate passing compilation configurations that closely resemble the failing compilation configuration. Specifically, these passing compilation configurations are designed to minimize differences from the failing compilation configurations, the differences in code coverage are minimized, allowing for better differentiation of suspicious files in the compiler that are responsible for the bug. To ensure minimal disparity, ODFL

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

generates passing compilation configurations based on the failing compilation configuration. In summary, ODFL adheres to the following two main principles:

- **Reduction Principle**: Failing configurations should disable as many fine-grained compilation options as possible.
- **Alignment Principle**: Passing configurations should closely resemble failing compilation configurations.

By following these two principles, ODFL can effectively isolate faulty files for compilers.

B. Key Steps

ODFL is comprised of three principal steps designed to derive distinct compilation configurations aimed at isolating compiler faults. These steps include: 1) Initial Configuration Selection, 2) Differentiated Fine-grained Configurations, and 3) Spectrum-Based Fault Localization. Below, we outline each step in detail.

1) Initial Configuration Selection: The first step involves selecting an initial compilation configuration for subsequent searches. Typically, when a user reports a compiler bug, the expected behavior is observed at lower levels of coarse-grained compilation. In contrast, unexpected behavior manifests at higher coarse-grained levels, where many fine-grained compilation options are enabled. To identify the specific failing configuration with the fewest fine-grained compilation options enabled, ODFL employs a search strategy that gradually disables each fine-grained compilation option, starting from a coarse-grained level.

While initializing the search from a relatively high coarse-grained compilation level may seem straightforward, the numerous fine-grained compilation options enabled by default at these levels make gradually disabling each one of them inefficient. To enhance efficiency, ODFL adopts a targeted approach by selecting the lowest coarse-grained compilation level that reproduces the same bug as the initial bug-triggering configuration. ODFL systematically evaluates each coarse-grained compilation level, progressing from lowest to highest, to identify the minimum level that reproduces the bug. This strategy minimizes the number of fine-grained compilation options enabled by default in the initial failing configuration, significantly reducing the search space for configurations with the fewest fine-grained compilation options enabled.

2) Differentiated Fine-Grained Configurations: In the second step, ODFL searches for differentiated compilation configurations. Initially, ODFL leverages compiler support to retrieve the set of fine-grained compilation options or compilation passes that are enabled by default in the initial compilation configuration selected in the first step, as described in Section IV-B1. Subsequently, ODFL searches for differentiated compilation configurations for GCC and LLVM as follows:

- **GCC:** In GCC, fine-grained compilation options are presented as unordered options. ODFL employs a step-by-step process to identify differentiated compilation configurations. Initially, it disables each fine-grained compilation option in the initial compilation configuration and observes the program's behavior. If disabling a specific fine-grained option successfully conceals the bug, it is identified as a bug-triggering option that directly contributes to the occurrence of the bug. Conversely, if disabling an option does not conceal the bug, it is categorized as a bug-free fine-grained option that is not responsible for the bug. The bug-free option is then disabled from the initial failing configuration to create the updated base failing configuration.

  Subsequently, ODFL iteratively disables the remaining bug-free fine-grained options one by one, starting from the updated base failing configuration and continuously updating it. This process continues until no further bug-free fine-grained options can be disabled among the remaining options. In other words, disabling any remaining fine-grained option must lead to the concealment of the bug. The objective is to derive a base failing configuration with minimal fine-grained options enabled. Finally, associated passing configurations are generated by independently disabling any of the bug-triggering options from the base failing configuration. Each passing configuration exhibits only one difference in enabled fine-grained options compared to the base failing configuration.
- **LLVM:** In LLVM, unlike GCC, fine-grained compilation options are provided as ordered passes. Since all LLVM passes are ordered, ODFL employs the binary search algorithm to identify the pass with the minimum index value that triggers the bug. By disabling passes with higher index values, ODFL obtains a failing configuration with minimal fine-grained options, which serves as the base failing configuration. The passing configurations are generated by selectively disabling passes with lower index values, while additional failing configurations are created by enabling extra passes with higher index values.

Ultimately, ODFL yields differentiated compilation configurations, consisting of the failing configuration with the fewest fine-grained compilation options enabled, along with the corresponding passing configurations that closely resemble the failing configuration in terms of the fine-grained compilation options or compilation passes enabled.

3) Spectrum-Based Fault Localization: Using the differentiated compilation configurations, specifically the adversary failing and passing configurations collected in the second step, as described in Section IV-B2, ODFL locates compiler faults employing the Spectrum-Based Fault Localization (SBFL) approach. In line with previous works [2], [3], ODFL utilizes the Ochiai formula [9], as defined in Formula 1, to compute the suspiciousness value for each touched statement s:

$$suspicious(s) = \frac{ef_s}{\sqrt{(ef_s + nf_s)(ef_s + ep_s)}} \tag{1}$$

where ef_s refers to the number of failing configurations that execute statement s, ep_s refers to the number of passing configurations that execute s, and nf_s refers to the number of failing configurations that do not execute s. The formula accounts for the number of failing and passing configurations that execute a statement, as well as the number of failing configurations that do not execute the statement.

With the suspicious values of each touched statement calculated, ODFL determines the suspicious value for each compiler

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Obtain the Differentiated Compilation Configurations

```
   Data: The compiler C, the program p, the failing and passing compilation level O_fail and O_pass
   Result: The set of failing configurations Confs_fail, the set of passing configurations Confs_pass
1  Procedure GetDifferentiatedConfigurations(C, p, O_fail, O_pass)
2      conf_initpass, conf_initfail ← GetInitialConfiguration(C, p, O_pass, O_fail)
3      FGO ← GetEnabledFineGrainedOptions(C, p, conf_initfail)
4      if FGO is ordered then
5          Confs_fail, Confs_pass ← Obtain the differentiated compilation configurations using binary search
6      else
7          Confs_fail, Confs_pass ← ∅, ∅
           /* Identify bug-triggering options                                          */
8          FGO_bt, FGO_left ← ∅, ∅
9          foreach fgo ∈ FGO do
10             conf_temp ← conf_initfail \ {fgo}
11             if Compile(C, p, conf_temp) == Fail then
12                 FGO_bt ← FGO_bt ∪ fgo
13             else
14                 FGO_left ← FGO_left ∪ fgo
           /* Obtain the base failing configuration with minimal fine-grained compilation options */
15         if Compile(C, p, conf_initfail \ FGO_left) == Fail then
16             conf_fail ← conf_initfail \ FGO_left
17         else
18             conf_curr ← conf_initfail
19             foreach fgo ∈ FGO_left do
20                 conf_temp ← conf_curr \ {fgo}
21                 if Compile(C, p, conf_temp) == Fail then
22                     conf_curr ← conf_temp
23             conf_fail ← conf_curr
24         Confs_fail ← Confs_fail ∪ Confs_fail
           /* Obtain differentiated compilation configurations                         */
25         foreach fgo ∈ FGO_bt do
26             conf_temp ← Confs_fail \ {fgo}
27             if Compile(C, p, conf_temp) == Pass then
28                 Confs_pass ← Confs_pass ∪ conf_temp
29             else
30                 Confs_fail ← Confs_fail ∪ conf_temp
31         if Confs_pass == ∅ then
32             Confs_pass ← {conf_initpass}
33     return (Confs_fail, Confs_pass)
   /* Get the adjacent differentiated coarse-grained compilation levels            */
34 Function GetInitialConfiguration(C, p, O_fail, O_pass)
35     foreach O_temp ∈ [O_pass, ..., O_fail] do
36         if Compile(C, p, O_temp) == Pass ∧ Compile(C, p, O_temp+1) == Fail then
37             return [O_temp, O_temp+1]
```

file. This is achieved by aggregating the suspicious values of all statements in a source file, using the formula established in previous studies [2], [3]. The formula is expressed as follows:

$$suspicious(f) = \frac{\sum_{i=1}^{n_f} suspicious(s_i)}{n_f} \tag{2}$$

where n_f is the number of statements covered by failing configurations in compiler file f. The files are then ranked by their suspicious values, with higher-ranked files being more likely to contain bugs.

4) Overall Algorithm: Algorithm 1 outlines the comprehensive procedure of ODFL for systematically searching for differentiated compilation configurations in a step-by-step manner. The algorithm begins with the selection of the lowest compilation level $conf_{initfail}$ that reproduces the bug with the same test program, serving as the initial compilation configuration (Line 2). The set of fine-grained compilation options enabled at $conf_{initfail}$ is denoted as FGO (Line 3).

Utilizing the chosen initial configuration and the fine-grained compilation options enabled by default, the algorithm proceeds to search for differentiated compilation configurations. In the

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

case of ordered fine-grained compilation options, such as those in LLVM, a binary search is employed (Lines 4-5). For compilers that utilize unordered fine-grained options, such as GCC, the algorithm aims to obtain a base failing configuration $conf_{fail}$ by systematically disabling as many fine-grained options as possible (Lines 8-22). Bug-triggering options are identified as FGO_{bt}, while the remaining options are classified as FGO_{left} (Lines 8-14). If disabling FGO_{left} from $conf_{initfail}$ still triggers the bug, the configuration is designated as $conf_{fail}$. Conversely, if the bug is no longer present, fine-grained options are gradually disabled from $conf_{initfail}$ to derive $conf_{fail}$ (Lines 18-22).

Passing compilation configurations $Confs_{pass}$ are obtained by individually disabling each fine-grained compilation option in FGO_{bt} from $conf_{fail}$ (Lines 25-30). By this way, each passing configuration in $Confs_{pass}$ exhibits only one difference in enabled fine-grained compilation options compared to $conf_{fail}$. Finally, the algorithm outputs the differentiated compilation configurations $Confs_{fail}$ and $Confs_{pass}$ (Line 33). This algorithmic approach facilitates the identification and exploration of fine-grained compilation options configurations, thereby aiding in the localization of compiler bugs.

C. Implementation

We have implemented ODFL as a practical tool for locating compiler bugs. Our implementation incorporates the algorithm outlined in Section IV-B4 to systematically search for differentiated compilation configurations, specifically the adversary failing and passing compilation configurations, for both GCC and LLVM compilers.

In the GCC compiler, each fine-grained compilation option is represented as an independent option, such as "-finline" and "-faligh-loops", which are enabled by default within the coarse-grained compilation level of "-O2". In contrast, LLVM organizes fine-grained compilation options as individual passes arranged in a specific order, allowing for a more structured approach to precisely control compilation during compilation.

To obtain the list of fine-grained compilation options enabled by default at a particular coarse-grained compilation level in GCC (e.g., "-O2"), we use the command "gcc -Q –help=optimizers -O2" to retrieve those options. Enabling or disabling individual options independently in GCC is straightforward; for instance, adding the flag "-fno-inline" disables the specific fine-grained compilation option "-finline".

In LLVM, we retrieve the list of fine-grained compilation passes enabled by default at a particular coarse-grained compilation level in LLVM (e.g., "-O2") using the command "clang -O2 -mllvm -opt-bisect-limit=-1". Unlike GCC, the ordered nature of fine-grained compilation passes in LLVM limits the flexibility to enable or disable individual options independently. However, LLVM provides the "-opt-bisect-limit=$\langle limit \rangle$" compilation debugging option, which allows users to precisely control the number of enabled passes. By specifying a limit value, compilation passes exceeding that limit can be disabled during code compilation, facilitating the identification of bugs caused by incorrect compilation passes that lead to erroneous runtime behavior. Through appropriate configuration of the limit value, specific bugs in LLVM can be effectively isolated.

In our implementation of ODFL, we utilize binary search to efficiently locate the threshold that triggers the bug in LLVM. Initially, we set the lower bound to 0 (disabling all passes) and the upper bound to the total number of enabled passes (enabling all fine-grained compilation passes), which can be obtained by setting the limit value to -1. It is important to note that different test programs with varying semantics will have different upper bounds for enabled passes. The program is then compiled using the middle value between the lower and upper bounds, and the compilation results are examined. If the compilation fails, the middle value becomes the new upper bound; otherwise, it becomes the new lower bound. This process continues until the lower and upper bounds converge, representing the threshold limit value that can trigger the bug. Specifically, if the limit value is set below this threshold, the bug will not be triggered; conversely, if it exceeds this threshold, the bug remains concealed.

By employing these strategies, ODFL facilitates the effective search for differentiated compilation configurations. These configurations adhere to the **Reduction Principle** and **Alignment Principle**, as described in Section IV-A. This approach significantly enhances the localization of compiler faults.

V. EVALUATION

In this section, we first summarize the research questions addressed in this study. Then, we describe the experimental setup, including the benchmarks, the baseline approaches, and the metrics used to evaluate compiler fault isolation techniques. Finally, we present the experimental results.

A. Research Questions

In this study, we address the following research questions:

- **RQ1:** (**Effectiveness**) How effective is ODFL in locating compiler faults?
- **RQ2:** (**Efficiency**) How efficient is ODFL in locating compiler faults?
- **RQ3:** (**Stability**) How do different SBFL formulas and flag order affect the performance of ODFL?

B. Experimental Setup

1) Subjects: To evaluate ODFL, we examine its effectiveness, efficiency, and stability in isolating bugs in GCC [15] and LLVM [16], which are widely used in both industry and academia [17], [18], [19], [20]. These compilers allow us to collect faulty files for each bug under investigation while offering precise control over fine-grained compilation options. Consistent with prior studies [2], [3], we use Gcov [21] to collect coverage information.

2) Benchmark: We use a benchmark consisting of 60 real GCC bugs and 50 real LLVM bugs, comparing ODFL against three state-of-the-art techniques: DiWi [2], RecBi [3], and

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

LLM4CBI [4]. A more recent technique, ETEM [22], introduces an enhanced program mutation strategy aimed at advancing the state of the art in compiler fault isolation. As described in its paper [22], ETEM "first searches for simplified optimization configurations that turn off fine-grained options as much as possible to narrow the coverage," effectively building upon the foundation established by our approach. Given that ETEM directly incorporates our technique as a core component of its method, we do not include it as a baseline in this study, as doing so would not provide an independent comparison.

The 60 GCC bugs were collected by Chen et al. [2], [3] and later used by Tu et al. [4]. We selected these bugs to ensure a fair comparison with previous techniques. For LLVM, 20 of the 60 bugs collected by Chen et al. [2], [3] are compatible with ODFL, as it relies on the "-opt-bisect-limit" feature introduced in LLVM version 267022 (April 22, 2016).[4] To expand the evaluation, we re-collected 30 additional LLVM bugs, resulting in a total of 50 LLVM bugs.

The selection of these 110 bugs follows specific criteria:

- All bugs have been resolved, enabling us to identify faulty code based on bug-fixing information.
- Most bugs were used as benchmarks in prior studies, facilitating direct comparisons.
- For each bug, we have detailed information about the compilation level that triggers the bug, the test program, the compiler version, and the faulty files. These faulty files serve as ground truth for evaluating different fault isolation approaches.

3) Baselines: To evaluate the effectiveness of ODFL in isolating compiler faults, we select DiWi [2], RecBi [3], and LLM4CBI [4] as baseline approaches:

- **DiWi:** DiWi [2] is a compiler bug isolation technique that generates effective witness test programs by mutating failing test cases. It focuses on creating diverse witness programs that share similar execution traces with the original program, thereby increasing the likelihood of eliminating non-contributing compiler files. This method significantly outperforms traditional approaches, establishing a robust baseline for the evaluation of other techniques.
- **RecBi:** RecBi [3] combines structural mutation with reinforcement learning to generate a set of effective passing test programs from a failing test program. The approach employs a reinforcement learning-based mutation process, incorporating both traditional local mutations and structural mutations to expand the mutation space and enhance the effectiveness of the generated test cases.
- **LLM4CBI:** LLM4CBI [4] leverages the capabilities of large language models (LLMs) to generate witness test programs for isolating compiler bugs. This approach involves generating and selecting effective prompts for LLMs to produce high-quality witness programs for compiler bug localization.

4) Measurements: A ranking list of suspicious compiler files is generated for each compiler bug by each compiler fault localization approach. We evaluate the effectiveness of compiler fault localization approaches by measuring the position of buggy files in the ranking lists. Since multiple compiler files may share the same suspicious value, we adopt the worst ranking as the final ranking for those files, in accordance with previous work [2], [3]. To assess the experimental results from various perspectives, we measure three key metrics that are widely utilized in prior research [2], [3], [23]:

- **Top-N**: This metric quantifies the number of bugs successfully located within the Top-N positions in the ranking list, where N is a member of the set $\{1, 5, 10, 20\}$. This selection aligns with prior studies [2], [3], [4]. A higher Top-N value indicates superior performance of the localization technique.
- **Mean First Rank (MFR)**: This metric computes the average rank of the first buggy element, specifically the buggy file in our study, within the ranking list for each bug. The purpose of the MFR metric is to evaluate how promptly the first buggy element is isolated. In debugging practice, the ranking position of the first buggy file is particularly important for developers. A smaller MFR value signifies that developers can localize the corresponding bug more quickly.
- **Mean Average Rank (MAR)**: This metric calculates the mean of the average rank of all buggy elements in the ranking list for each bug. The MAR metric evaluates the average ranking performance for all faulty elements for each bug. Similar to MFR, a smaller MAR indicates a better performance of the localization approach.

C. RQ1: Effectiveness of ODFL

1) Experimental Settings: To evaluate the effectiveness of ODFL, we compare it with three established techniques: DiWi [2], RecBi [3], and LLM4CBI [4]. The effectiveness of these techniques will be assessed in terms of their ability to locate GCC and LLVM compiler bugs, respectively.

For the evaluation of GCC compiler bugs, we use a set of 60 GCC bugs to compare ODFL with DiWi, RecBi, and LLM4CBI. These 60 bugs have also been employed in prior studies evaluating DiWi, RecBi, and LLM4CBI, allowing for a direct comparison of the effectiveness of ODFL with these existing techniques in identifying faults within GCC compilers. DiWi, RecBi, and LLM4CBI are techniques based on generating witness test programs, which involve elements of randomness during the mutation of existing bug-triggering test cases. To mitigate the influence of randomness when replicating these techniques, this study directly utilizes the results from previous research that employed the same set of 60 GCC bugs. This ensures a direct and equitable comparison with the state-of-the-art techniques. Here, we adopt the same experimental setup as in all prior studies [2], [3], [4]. Specifically, each technique generates mutants continuously until exceeding one hour of execution time. Additionally, each technique is executed five times, with the best result recorded as the measure of effectiveness. Moreover, to ensure a fair comparison, we use the reported results from Tu et al.'s recent study under the default

[4]https://github.com/llvm/llvm-project/commit/f0f2792

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EFFECTIVENESS COMPARISON FOR ODFL VS. THE BASELINE APPROACHES

Subject	Technique	Top-1	$\Uparrow_{Top-1}$	Top-5	$\Uparrow_{Top-5}$	Top-10	$\Uparrow_{Top-10}$	Top-20	$\Uparrow_{Top-20}$	MFR	$\Uparrow_{MFR}$	MAR	$\Uparrow_{MAR}$
GCC	DiWi	5.60	221.43%	19.90	60.80%	31.30	30.99%	41.70	12.71%	22.57	-36.51%	23.10	-53.18%
	RecBi	7.70	133.76%	23.90	33.89%	34.20	19.88%	42.50	10.59%	20.53	-30.20%	21.06	-28.40%
	LLM4CBI	9.30	93.55%	24.90	28.51%	36.40	12.64%	44.80	4.91%	15.84	-9.53%	16.35	-7.77%
	ODFL	**18.00**	—	**32.00**	—	**41.00**	—	**47.00**	—	**14.33**	—	**15.08**	—
LLVM	DiWi	4.00	225.00%	15.00	60.00%	20.00	60.00%	31.00	32.26%	24.92	-49.44%	26.05	-51.17%
	RecBi	5.00	160.00%	16.00	50.00%	22.00	45.45%	33.00	24.24%	26.88	-53.13%	27.92	-54.44%
	ODFL	**13.00**	—	**24.00**	—	**32.00**	—	**41.00**	—	**12.60**	—	**12.72**	—
ALL	DiWi	9.60	222.92%	34.90	60.46%	51.30	42.30%	72.70	21.05%	23.64	-42.72%	24.44	-42.68%
	RecBi	12.70	144.09%	39.90	40.35%	56.20	29.89%	75.50	16.56%	23.42	-42.19%	24.18	-42.06%
	ODFL	**31.00**	—	**56.00**	—	**73.00**	—	**88.00**	—	**13.54**	—	**14.01**	—

Columns "$\Uparrow_*$" refers to the improvement rate of ODFL over a compared technique in terms of the * measurement.

setting (i.e., one-hour termination) as the effectiveness metrics for DiWi, RecBi, and LLM4CBI in our comparison with ODFL.

For the evaluation of LLVM compiler bugs, we utilize a total of 50 LLVM compiler bugs to compare ODFL with DiWi and RecBi. As LLM4CBI requires large language models for generating witness test programs, we limit our comparison of ODFL to DiWi and RecBi in locating the LLVM compiler bugs, excluding LLM4CBI from this analysis. We made this choice because, according to their evaluations, LLM4CBI does not demonstrate a substantial improvement over DiWi and RecBi in terms of the number of bugs successfully identified; it can only locate about 2 more bugs than RecBi out of the 60 LLVM bugs within the Top-1 ranked files.

2) Experimental Results:

Effectiveness. The rows DiWi, RecBi, LLM4CBI, and ODFL in Table I demonstrate the respective effectiveness of these methods on bugs in the GCC and LLVM compilers. To ensure a fair comparison and mitigate the influence of random factors, such as those introduced by mutations and LLMs, the results for DiWi [2], RecBi [3], and LLM4CBI [4] on the GCC bug dataset are directly cited from their respective publications. Since this study employs a new LLVM bug benchmark, the DiWi and RecBi methods were re-evaluated on this dataset. Additionally, the related work [4] acknowledges that LLM4CBI exhibits similar performance to RecBi on the LLVM bug dataset. Therefore, in the experiment involving LLVM bugs, we compare ODFL only against DiWi and RecBi, using their results to represent the performance of LLM4CBI.

Overall, ODFL locates 31, 56, 73, and 88 bugs out of a total of 110 bugs within Top-1, Top-5, Top-10, and Top-20 files, respectively. This corresponds to 28%, 51%, 66%, and 80% effectiveness. ODFL has MFR and MAR values of 13.54 and 14.01, respectively, meaning it locates the first buggy file within the top 13.54 files, and all buggy files within the top 14.01 files, on average. Specifically for GCC, ODFL locates 18, 32, 41, and 47 bugs within the Top-1, Top-5, Top-10, and Top-20 files, respectively. For LLVM, ODFL locates 13, 24, 32, and 41 bugs within Top-1, Top-5, Top-10, and Top-20 files, respectively. Additionally, the comparison of ODFL with alternative techniques indicates that ODFL enhances compiler fault localization when compared to previous techniques. For instance, ODFL localizes 144.09%, 40.35%, 29.89%, and 16.56% more faults than RecBi within the Top-1, Top-5, Top-10, and Top-20 files, respectively. Furthermore, ODFL exceeds RecBi by 42.19% and 42.06% in terms of MFR and MAR, respectively. This suggests that ODFL achieves more precise localization outcomes, signifying its superior effectiveness relative to the state-of-the-art approaches.

Statistical Test Analysis of Results. In line with the existing work [4], we conduct a Mann-Whitney U-test with a significance level of 0.05 on the total bugs between ODFL and the state-of-the-art approaches, following the recommendations of Arcuri and Briand [24].

To mitigate the influence of randomness, we also calculate the effect size of the differences between ODFL and the baseline techniques using Vargha and Delaney's $\hat{A}_{12}$ statistics. In this context, given a performance measure M (e.g., Top-1), the $\hat{A}_{12}$ statistic quantifies the probability that approach A (e.g., ODFL) yields higher M values than approach B (e.g., DiWi). If the two approaches are equivalent, then $\hat{A}_{12} = 0.5$. For instance, $\hat{A}_{12} = 0.9$ indicates that approach A produces higher results 90% of the time compared to approach B.

We utilize the R programming language to perform the Mann-Whitney U-test and obtain the p-value. For calculating the effect size, we employ the formula:

$$\hat{A}_{12} = \frac{(R1/m) - \left(\frac{m+1}{2}\right)}{n}$$

where $R1$ is the rank sum of the first data group being compared and m represents the number of observations in the first group and n represents the number of observations in the second group. The rank sum is a fundamental component of the Mann-Whitney U-test and is provided by most

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

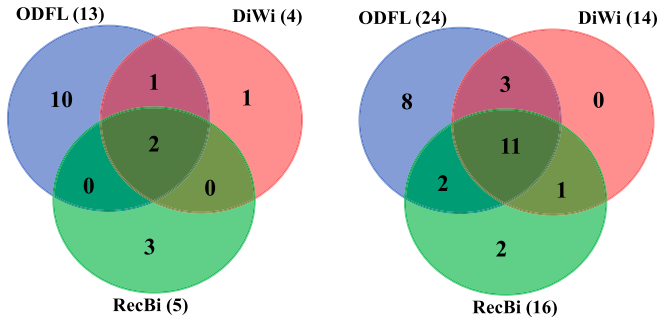
TABLE II
STATISTIC TEST RESULTS FOR ODFL VS STATE-OF-THE-ART TECHNIQUES IN TERMS OF LOCATING GCC BUGS

Comparison	Metric	p-value	$\hat{A}_{12}$
ODFL vs. DiWi	Top-1	< 0.001	1.0
	Top-5	< 0.001	1.0
	Top-10	< 0.001	1.0
	Top-20	< 0.001	1.0
	MFR	< 0.001	1.0
	MAR	< 0.001	1.0
ODFL vs. RecBi	Top-1	< 0.001	1.0
	Top-5	< 0.001	1.0
	Top-10	< 0.001	1.0
	Top-20	< 0.001	1.0
	MFR	< 0.001	1.0
	MAR	< 0.001	1.0
ODFL vs. LLM4CBI	Top-1	< 0.001	1.0
	Top-5	< 0.001	1.0
	Top-10	< 0.001	0.9
	Top-20	0.016	0.8
	MFR	< 0.001	1.0
	MAR	0.001	0.9

statistical tools. We compute the $R1$ value using the function "$sum(rank(c(X, Y))[seq_along(X)])$" in R, where X and Y are the datasets containing the observations (i.e., the metrics over GCC bugs) of the two approaches being compared.

The overall statistical results for both experimental settings are presented in Table II. A p-value less than 0.05 (often less than 0.001) indicates that ODFL performs significantly better than all existing approaches, namely DiWi [2], RecBi [3], and LLM4CBI [4]. Furthermore, we observe that all effect sizes are greater than 0.8 for both techniques, indicating that ODFL has an extremely high probability of yielding better results than the two state-of-the-art approaches.

Minimal Set of Essential Optimization Flags. We further analyze the number of optimization flags involved during fault localization based on the dataset. Table IV presents the number of compiler flags enabled under four commonly used optimization levels. On average, `-O1` enables 77 flags, while `-O2` and `-Os` activate substantially more, averaging 108 flags. The `-O3` level enables the most, with an average of 115 flags, reflecting its focus on aggressive optimizations. These results underscore the substantial complexity of default optimization settings and the need for efficient configuration reduction strategies. Table V reports the average number of essential flags required to trigger compiler bugs in the dataset. Across all evaluated cases, only 5 essential flags are needed on average, just 4.4% of the 114 flags typically enabled by default. This means that fault localization often requires disabling approximately 109 additional flags, with the exact number ranging from 67 to 133 depending on the optimization level and compiler version. Notably, in the most extreme cases, ODFL isolates bugs triggered by a single essential flag, while disabling up to 133 extraneous flags. This



(a) Top-1 (b) Top-5

Fig. 5. Distribution of correct localization on LLVM achieved by ODFL and baselines in terms of Top-1 and Top-5.

stark contrast between the default optimization space and the minimal triggering configurations highlights ODFL's precision and effectiveness in isolating fault-inducing compiler options.

Unique Capability. To further examine ODFL's unique capability, we analyze the distribution of correctly localized LLVM faults by ODFL and the baseline methods using the Top-1 and Top-5 indicators. The results are presented in Fig. 5. Specifically, as shown in Fig. 5(a), 10 out of the 13 faults localized by ODFL at Top-1 are not identified by any of the baselines. This underscores ODFL's distinctive ability in localizing faults that existing methods fail to detect.

Moreover, ODFL demonstrates broader coverage by successfully localizing most of the faults identified by RecBi and DiWi at Top-5, while also uncovering additional faults unique to ODFL. These findings collectively highlight ODFL's originality and effectiveness in compiler fault localization. We conclude that ODFL offers a specialized and highly effective solution for this challenging task.

In addition, we observed that the existing techniques based on witness test programs are able to localize a small number of unique faults not detected by ODFL, suggesting potential opportunities to combine multiple methods to further enhance compiler fault isolation. However, such combinations also entail certain limitations. First, the computational cost of localization may increase significantly due to the overhead introduced by program mutations. Second, divergent top-ranked results across different techniques may lead to a decreased performance and transfer the manual inspection burden onto users rather than alleviating it.

D. RQ2: Efficiency of ODFL

1) Experimental Setup: To evaluate efficiency, we compare the execution time required by ODFL with that of RecBi, DiWi, and LLM4CBI. Our focus is on the time needed to generate differentiated configurations in ODFL and the time required to generate witness test programs in the baseline methods. For the baseline techniques, we adopted the respective settings originally used in their published papers. Specifically, both DiWi and RecBi were executed to generate mutants within a one-hour time frame. It is important to note that we did not include the time required to obtain code coverage for the compilers across

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EFFICIENCY COMPARISON BETWEEN ODFL VS. THE BASELINES APPROACHES IN TERMS OF TIME-CONSUMPTION (HOURS)

Technique	GCC						LLVM					
	Avg	$\Downarrow_{Avg}$	**Max**	$\Downarrow_{Max}$	**Min**	$\Downarrow_{Min}$	**Avg**	$\Downarrow_{Avg}$	**Max**	$\Downarrow_{Max}$	**Min**	$\Downarrow_{Min}$
`RecBi`	1.0*h*	99.65%	1.0*h*	99.34%	1.0*h*	99.81%	1.0*h*	99.89%	1.0*h*	99.83%	1.0*h*	99.92%
`DiWi`	1.0*h*	99.65%	1.0*h*	99.34%	1.0*h*	99.81%	1.0*h*	99.89%	1.0*h*	99.83%	1.0*h*	99.92%
`LLM4CBI`	∼0.4*h*	99.12%	∼0.6*h*	98.90%	∼0.3*h*	99.37%	∼0.4*h*	99.73%	∼0.7*h*	99.76%	∼0.3*h*	99.73%
ODFL	**0.0035*h***	—	**0.0066*h***	—	**0.0019*h***	—	**0.0011*h***	—	**0.0017*h***	—	**0.0008*h***	—

TABLE IV
NUMBER OF FLAGS ENABLED BY EACH OPTIMIZATION LEVEL

Optimization Level	Average	Max	Min
`-O1`	77	88	67
`-O2`	108	116	99
`-O3`	115	133	104
`-Os`	108	119	101

TABLE V
STATISTICS OF OPTIMIZATION FLAGS IN GCC BUG ISOLATION

Category	Average	Max	Min
Essential flags for bug triggering	5	13	1
Flags enabled by default	114	141	78
Additional flags to disable	109	133	67

TABLE VI
THE SIX ADAPTED SBFL FORMULAE IN ODFL

Name	Formula
Ochiai [9]	$\frac{ef_s}{\sqrt{(ef_s+nf_s)(ef_s+ep_s)}}$
Tarantula [28]	$\frac{ef_s/totalf_s}{ef_s/totalf_s+ep_s/totalp_s}$
Ochiai2 [29]	$\frac{ef_s np_s}{\sqrt{(ef_s+ep_s)(nf_s+np_s)(ef_s+np_s)(nf_s+ep_s)}}$
Op2 [29]	$ef_s - \frac{ep_s}{(totalp_s+1)}$
Barinel [5]	$1 - \frac{ep_s}{ep_s+ef_s}$
DStar [30]	$\frac{ef_s^\star}{ep_s+nf_s}$

We used $\star = 2$, the most thoroughly explored value [26].

all techniques, as we assume that this coverage acquisition time remains consistent across methods. For LLM4CBI, Tu et al. used the time required to generate ten passing test programs as a measure of efficiency, where it typically requires less than one hour on average to produce these programs. Therefore, to maintain consistency with the original settings, we adopt this in our comparison.

2) Experiment Results: Table III summarizes the results for our efficiency evaluation. As shown, ODFL can generate differentiated configurations for GCC within an average timeframe of 0.0035 hours and for LLVM within an average timeframe of 0.0011 hours. In contrast, RecBi and DiWi require 1 hour to mutate a sufficient number of witness test programs, while LLM4CBI requires around 0.4 hours for the same task.

Therefore, compared to the state-of-the-art compiler fault localization approaches, ODFL demonstrates over a 99% improvement in efficiency for both GCC and LLVM. This significant enhancement in efficiency underscores ODFL's practicality in aiding compiler fault localization.

E. RQ3: Stability of ODFL

1) Experimental Setup: ODFL defaults to using the Ochiai formula [9] to calculate the suspiciousness values of statements in the compiler, as it has been reported to be the most effective SBFL formula [25], [26] and has also been employed in DiWi, RecBi, and LLM4CBI. However, it remains unclear whether ODFL can achieve comparable performance using other SBFL formulas. In this study, we compare Ochiai with five other widely studied SBFL formulae [25], [26], [27] to investigate their impact on ODFL's performance. Table VI presents the SBFL formulae we examined. In these formulae, ef_s, nf_s, and ep_s are defined as in Formula 1; np_s denotes the number of passing cases that do not execute statement s, while $totalf_s$ and $totalp_s$ represent the total number of failing and passing cases for s, respectively.

Another factor potentially affecting ODFL's performance is the order in which GCC's fine-grained optimization options are disabled to generate compilation configurations. By default, ODFL disables these options in the fixed, compiler-defined order. For example, under the `-O2` optimization level, we retrieve the list of fine-grained options via the command `gcc -Q help=optimizers -O2`, which outputs the options in GCC's predetermined order. ODFL then sequentially disables each option according to this list. However, varying the disabling order can yield different outcomes. To assess ODFL's robustness to such permutations, we conducted ten independent trials in which the GCC options were disabled in a randomized order for each run. This experiment evaluates whether ODFL's performance remains consistent under randomized disabling sequences.

2) Experimental Results: To examine the stability of ODFL across different SBFL formulae, we evaluated its performance

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
STABILITY OF ODFL UNDER DIFFERENT FORMULAE

Subject	Formula	Top-1	Top-5	Top-10	Top-20	MAR	MFR
GCC	**Ochiai**	18	**32**	41	47	14.33	15.08
	Tarantula	14	**32**	39	**49**	10.63	**11.38**
	Ochiai2	14	26	36	46	12.68	13.41
	Op2	16	27	40	46	26.88	27.82
	Barinel	18	**32**	38	48	**10.58**	11.54
	DStar	**20**	31	**42**	47	23.05	23.78
LLVM	**Ochiai**	13	**24**	**32**	41	12.60	12.72
	Tarantula	**14**	23	**32**	**42**	**10.86**	**11.03**
	Ochiai2	5	6	7	12	556.54	565.01
	Op2	7	16	20	27	308.68	309.45
	Barinel	11	20	28	**42**	11.58	11.76
	DStar	**14**	21	29	38	85.88	86.35

using standard fault localization metrics, including Top-1, Top-5, Top-10, Top-20, MAR, and MFR. Table VII summarizes the results for each SBFL formula, where rows represent formulae and columns correspond to evaluation metrics. Overall, the results indicate that ODFL exhibits stability across most SBFL formulae. Notably, the adoption of the DStar formula yields relatively optimal performance for the Top-1 and Top-10 metrics in GCC, as shown in Table VII. Similarly, employing the Tarantula formula achieves the best performance for the Top-1, Top-10, and Top-20 metrics in LLVM. In GCC, while different formulae result in varying performances, ODFL consistently achieves significant improvements compared to DiWi, RecBi, and LLM4CBI across all metrics. In contrast, in LLVM, the choice of formula has a more pronounced impact on ODFL's stability. However, ODFL outperforms existing techniques in most cases. These findings highlight that while different formulae influence ODFL's performance to some extent, ODFL remains robust and generally surpasses existing approaches across the evaluated metrics.

To further assess ODFL's stability, we evaluated its performance across ten trials, each using a randomized sequence for disabling GCC's fine-grained optimization options. Table VIII presents the results, showing the performance metrics for each trial. Randomizing the order for disabling GCC options introduced slight variations in performance. For instance, the Top-1 accuracy ranged from 12 to 15, while the Top-5 accuracy varied between 28 and 31. Despite these fluctuations, the overall differences were minimal, demonstrating that ODFL remains largely stable even when the disabling order is randomized. Nevertheless, due to dependencies among certain options, such as `-frename-registers`, which is enabled by default when `-funroll-loops`[5] is active, the original order defined within compiler (adopted as ODFL's default order) consistently yields the best performance.

[5]https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

TABLE VIII
STABILITY OF ODFL UNDER RANDOMIZED CONDITIONS

Trial	Top-1	Top-5	Top-10	Top-20	MFR	MAR
1	14	28	47	53	11.78	14.11
2	13	29	45	51	9.32	11.64
3	12	30	45	53	11.42	12.19
4	14	31	46	53	8.62	9.39
5	13	29	47	53	8.58	10.91
6	14	29	44	51	13.25	15.57
7	14	30	45	52	13.00	13.78
8	14	30	45	53	9.12	11.44
9	13	31	47	52	13.27	15.59
10	15	31	46	52	12.63	14.98

VI. THREATS TO VALIDITY

Our work has the following potential threats to validity.

First, while our algorithm efficiently identifies failing configurations that enable minimal compilation options in most cases, it may fail to generate corresponding passing configurations in certain scenarios. This limitation can arise when finer-grained passing configurations are not detectable due to the complex interplay of compilation options and bug-triggering conditions. To mitigate this, we adopt the highest bug-concealing compilation level as the passing configuration when no finer-grained passing configuration can be identified. For example, if a bug is not triggered at -O0 or -O1 but is triggered at -O2, we designate -O1 as the passing configuration if a more precise alternative is unavailable.

Second, to ensure a fair and comprehensive evaluation of fault localization techniques, we extended the benchmarks by combining the 80 bugs from prior studies [2], [3] (60 GCC bugs and 20 LLVM bugs) with an additional 30 LLVM bugs that we collected. While this enhances the robustness of our experimental results and mitigates the risks posed by a limited dataset, potential biases may still exist in the selection and distribution of bugs. For example, the bugs from prior studies may reflect specific characteristics or patterns that differ from those in our newly collected bugs, potentially influencing the generalizability of our findings. Additionally, the quality and completeness of the bug reports may vary across the dataset, potentially affecting the accuracy of our evaluations.

VII. RELATED WORK

Our study focuses on the topic of software fault localization. The literature presents a substantial body of research in this area, including spectrum-based [5], [6], [27], slicing-based [31], [32], mutation-based [33], [34], [35], and model-based fault localization [36], [37]. Among these, spectrum-based approaches [5], [6] are particularly prevalent. These methods calculate the suspiciousness of code elements by comparing the coverage of failing and passing test cases. Unfortunately, due to the complexity of compilers, obtaining effective passing coverage capable of isolating innocent code elements responsible for compiler bugs remains a significant challenge.

Recently, Chen et al. [2] were the first to highlight this critical issue and introduced the concept of the witness test program

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

to facilitate compiler fault isolation. A witness test program is derived from existing bug-triggering test programs and does not manifest the bug itself. By utilizing both the initial bug-triggering test program and the generated witness test program, it becomes possible to obtain the failing and passing coverage necessary for isolating compiler faults. The SBFL formula is then employed to calculate the suspiciousness scores of the code elements implicated in the faults.

Subsequently, they proposed a reinforcement learning-based approach, RecBi [3], which combines structural mutation with reinforcement learning to generate a set of effective witness test programs from bug-triggering test programs. This approach incorporates a reinforcement learning-based mutation process that integrates both traditional local mutations and structural mutations, thereby expanding the mutation space and enhancing the effectiveness of the generated test cases.

Following this, Tu et al. [4] introduced LLM4CBI, which utilizes Large Language Models (LLMs) to refine the generation of witness test programs. In general, DiWi, RecBi, and LLM4CBI focus on producing sets of effective passing test programs that can eliminate suspicion of innocent compiler files. However, these methods still face limitations regarding their effectiveness and efficiency.

In response, we propose ODFL, a novel compiler fault localization approach that utilizes differentiated compilation configurations. While existing techniques typically generate failing test cases from the perspective of input test programs, our method constructs failing test cases by adjusting compilation configurations. Experimental results demonstrate that ODFL significantly outperforms DiWi, RecBi, and LLM4CBI, with all SBFL formulae yielding similar results, further demonstrating the effectiveness of our approach based on differentiated configurations. The impact of ODFL has already begun to influence subsequent research. For instance, a recent technique, ETEM [22], integrates ODFL with mutation-based strategies to enhance the isolation of compiler bugs. While ETEM offers enhanced capabilities in isolating compiler bugs, it also introduces trade-offs in terms of computational efficiency.

Additionally, there is research related to automated compiler debugging. For instance, several studies have focused on minimizing failing test programs to facilitate debugging [38], [39], [40], [41]. Holmes and Groce introduced new metrics to measure the distance between failing test programs, which aids in compiler debugging [42], [43]. In contrast, our work specifically concentrates on configuring fine-grained compilation options to effectively locate compiler bugs.

VIII. CONCLUSION

In this study, we introduce a novel concept of differentiated compilation configurations which consist of failing and passing configurations. The failing configurations enable the minimum number of fine-grained compilation options necessary to trigger the bug, while the passing configuration closely resembles the failing configurations. We propose ODFL, a novel compiler fault localization technique by utilizing differentiated compilation configurations. Our experimental results demonstrate that ODFL significantly outperforms the state-of-the-art compiler fault localization approaches in terms of both effectiveness and efficiency.

DATA AVAILABILITY STATEMENT

The scripts associated with ODFL used in this study are publicly available at https://doi.org/10.5281/zenodo.15375231.

ACKNOWLEDGMENT

We would like to express our appreciation to the authors of DiWi, RecBi, and LLM4CBI for generously sharing their tools and data. We are also grateful to the anonymous reviewers and to Professor Tracy Hall for their valuable and insightful feedback. Additionally, we are indebted to Professor Ming Wen and Professor Hai Jin for their contributions to the conference version of this work.

REFERENCES

[1] Z. Zhou, Z. Ren, G. Gao, and H. Jiang, "An empirical study of optimization bugs in GCC and LLVM," *J. Syst. Softw.*, vol. 174, Apr. 2021, Art. no. 110884, doi: 10.1016/j.jss.2020.110884.
[2] J. Chen, J. Han, P. Sun, L. Zhang, D. Hao, and L. Zhang, "Compiler bug isolation via effective witness test program generation," in *Proc. ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/SIGSOFT FSE)*, Tallinn, Estonia, 2019, pp. 223–234.
[3] J. Chen, H. Ma, and L. Zhang, "Enhanced compiler bug isolation via memoized search," in *Proc. 35th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Melbourne, Australia, 2020, pp. 78–89.
[4] H. Tu, Z. Zhou, H. Jiang, I. N. B. Yusuf, Y. Li, and L. Jiang, "Isolating compiler bugs by generating effective witness programs with large language models," *IEEE Trans. Softw. Eng.*, vol. 50, no. 7, pp. 1768–1788, Jul. 2024, doi: 10.1109/TSE.2024.3397822.
[5] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "Spectrum-based multiple fault localization," in *Proc. 24th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Auckland, New Zealand, 2009, pp. 88–99.
[6] C. M. Tang, W. K. Chan, Y. Yu, and Z. Zhang, "Accuracy graphs of spectrum-based fault localization formulas," *IEEE Trans. Rel.*, vol. 66, no. 2, pp. 403–424, Jun. 2017.
[7] X. Xie, T. Y. Chen, F.-C. Kuo, and B. Xu, "A theoretical analysis of the risk evaluation formulas for spectrum-based fault localization," *ACM Trans. Softw. Eng. Methodol.*, vol. 22, no. 4, Oct. 2013, Art. no. 31, doi: 10.1145/2522920.2522924.
[8] J. Yang, Y. Yang, M. Sun, M. Wen, Y. Zhou, and H. Jin, "Isolating compiler optimization faults via differentiating finer-grained options," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. ReEng. (SANER)*, Honolulu, HI, USA, Piscataway, NJ, USA: IEEE, 2022, pp. 481–491, doi: 10.1109/SANER53432.2022.00065.
[9] R. Abreu, P. Zoeteweij, and A. J. Van Gemund, "On the accuracy of spectrum-based fault localization," in *Proc. Test., Acad. Ind. Conf. Pract. Res. Techn.-MUTATION (TAICPART-MUTATION)*, 2007, pp. 89–98.
[10] F. Keller, L. Grunske, S. Heiden, A. Filieri, A. van Hoorn, and D. Lo, "A critical evaluation of spectrum-based fault localization techniques on a large-scale software system," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. (QRS)*, Prague, Czech Republic, 2017, pp. 114–125.
[11] C. Sun, V. Le, Q. Zhang, and Z. Su, "Toward understanding compiler bugs in GCC and LLVM," in *Proc. 25th Int. Symp. Softw. Test. Anal. (ISSTA)*, New York, NY, USA: ACM, 2016, pp. 294–305, doi: 10.1145/2931037.2931074
[12] P. B. Schneck, "A survey of compiler optimization techniques," in *Proc. ACM Annu. Conf.*, New York, NY, USA: ACM, 1973, pp. 106–113, doi: 10.1145/800192.805690.
[13] V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers Principles, Techniques & Tools. (2nd Edition) USA: Addison-Wesley Longman Publishing Co., Inc., 2006.
[14] Z. Zhou, Z. Ren, G. Gao, and H. Jiang, "An empirical study of optimization bugs in GCC and LLVM," *J. Syst. Softw.*, vol. 174, 2021, Art. no. 110884.
[15] "GCC." Accessed: 2023. [Online]. Available: https://gcc.gnu.org/
[16] "LLVM." Accessed: 2023. [Online]. Available: https://llvm.org/

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.

[17] T. Glek and J. Hubicka, "Optimizing real world applications with GCC link time optimization," 2010, *arXiv:1010.2196*.
[18] G. Fursin et al., "Milepost GCC: Machine learning based research compiler," in *Proc. GCC Developers' Summit*, 2008, pp. 1–13.
[19] L. P. Cáceres, F. Pagnozzi, A. Franzin, and T. Stützle, "Automatic configuration of GCC using Irace," in *Proc. 13th Int. Conf. Artif. Evol., Évolution Artificielle (EA)*, Paris, France, 2017, pp. 202–216.
[20] D. Berlin, D. Edelsohn, and S. Pop, "High-level loop optimizations for GCC," in *Proc. GCC Develop. Summit*, 2004, pp. 37–54.
[21] "GCOV." Accessed: 2023. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Gcov.html
[22] Y. Liu, M. Zhu, J. Dong, J. Yu, and D. Hao, "Compiler bug isolation via enhanced test program mutation," in *Proc. 39th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2024, pp. 819–830, doi: 10.1145/3691620.3695074.
[23] J. Sohn and S. Yoo, "FLUCCS: Using code and change metrics to improve fault localization," in *Proc. 26th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Santa Barbara, CA, USA, 2017, pp. 273–283.
[24] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proc. 33rd Int. Conf. Softw. Eng. (ICSE)*, 2011, pp. 1–10.
[25] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016.
[26] S. Pearson et al., "Evaluating and improving fault localization," in *Proc. 39th Int. Conf. Softw. Eng. (ICSE)*, Buenos Aires, Argentina, 2017, pp. 609–620.
[27] M. Wen et al., "Historical spectrum based fault localization," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2348–2368, Nov. 2021.
[28] J. A. Jones and M. J. Harrold, "Empirical evaluation of the tarantula automatic fault-localization technique," in *Proc. 20th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Long Beach, CA, USA, 2005, pp. 273–282.
[29] L. Naish, H. J. Lee, and K. Ramamohanarao, "A model for spectra-based software diagnosis," *ACM Trans. Softw. Eng. Methodol.*, vol. 20, no. 3, pp. 11:1–11:32, 2011.
[30] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The DStar method for effective software fault localization," *IEEE Trans. Rel.*, vol. 63, no. 1, pp. 290–308, Mar. 2014.
[31] H. Agrawal, R. A. DeMillo, and E. H. Spafford, "Debugging with dynamic slicing and backtracking," *Softw. Pract. Exp.*, vol. 23, no. 6, pp. 589–616, 1993.
[32] X. Zhang, H. He, N. Gupta, and R. Gupta, "Experimental evaluation of using dynamic slices for fault location," in *Proc. 6th Int. Workshop Automat. Debugging (AADEBUG)*, Monterey, CA, USA, 2005, pp. 33–42.
[33] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *Proc. 7th IEEE Int. Conf. Softw. Test., Verif. Validation (ICST)*, Cleveland, OH, USA, 2014, pp. 153–162.
[34] S. Hong et al., "Mutation-based fault localization for real-world multilingual programs (T)," in *Proc. 30th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Lincoln, NE, USA, 2015, pp. 464–475.
[35] M. Papadakis and Y. L. Traon, "Metallaxis-FL: Mutation-based fault localization," *Softw. Test. Verif. Rel.*, vol. 25, nos. 5–7, pp. 605–628, 2015.
[36] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "An observation-based model for fault localization," in *Proc. Int. Workshop Dyn. Anal. (WODA), Held Conjunction ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA)*, Seattle, WA, USA, 2008, pp. 64–70.
[37] M. Wen, R. Wu, and S. Cheung, "Locus: Locating bugs from software changes," in *Proc. 31st IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Singapore, 2016, pp. 262–273.
[38] J. M. Caron and P. A. Darnell, "Bugfind: A tool for debugging optimizing compilers," *ACM SIGPLAN Notices*, vol. 25, no. 1, pp. 17–22, 1990.
[39] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Mar. 2005.
[40] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang, "Test-case reduction for C compiler bugs," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implementation (PLDI)*, Beijing, China, 2012, pp. 335–346.
[41] S. Herfert, J. Patra, and M. Pradel, "Automatically reducing tree-structured test inputs," in *Proc. 32nd IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Urbana, IL, USA, 2017, pp. 861–871.
[42] J. Holmes and A. Groce, "Causal distance-metric-based assistance for debugging after compiler fuzzing," in *Proc. 29th IEEE Int. Symp. Softw. Rel. Eng. (ISSRE)*, Memphis, TN, USA, 2018, pp. 166–177.
[43] J. Holmes and A. Groce, "Using mutants to help developers distinguish and debug (compiler) faults," *Softw. Test., Verif. Rel.*, vol. 30, no. 2, 2020, Art. no. e1727.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:13:30 UTC from IEEE Xplore. Restrictions apply.